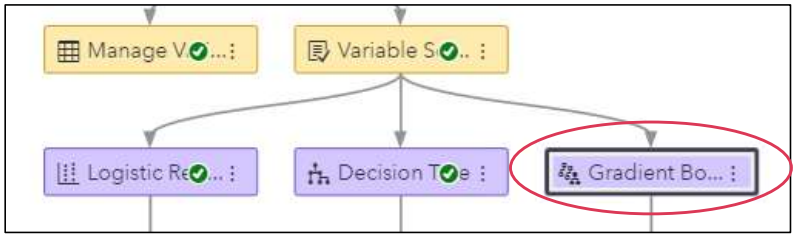
Building a Gradient Boosting Model

The algorithm for gradient boosting evolved from the application of boosting methods to regression trees. The main idea is to compute a sequence of simple trees, where each successive tree is built for the prediction residuals of the preceding tree. This method builds trees by partitioning the data into samples at each split node. Then, at each step of the boosting trees algorithm, a best partitioning of the data is determined, and the deviations of the observed values from the respective residuals for each partition are computed. The next trees are fitted to those residuals, to find another partition that further reduces the residual variance for the data, given the preceding sequence of trees.

This additive weighted approach of trees can produce excellent fit of the predicted values to the observed values, even if the specific nature of the relationships between the inputs and the target is complex. For that reason, the method of gradient boosting by fitting a weighted additive expansion of simple trees can create general and powerful machine learning models.

In this demonstration, you add a Gradient Boosting node to the Lesson 3 pipeline. You build a default gradient boosting model, change some of the settings, and compare the model to the other models in the pipeline.

1. In the Lesson 3 pipeline, right-click the **Variable Selection** node and select **Add child node** ⇨ **Supervised Learning** ⇨ **Gradient Boosting**.



2. Keep all properties for the Gradient Boosting node at their defaults. Run the **Gradient Boosting** node.
3. Open the results for the Gradient Boosting node.
4. Click the **Assessment** tab.

Target ...	Data Role	Partitio...	Formatt...	Sum of ...	Averag...
churn	TRAIN	1	1	39,590	0.0546
churn	VALIDATE	0	0	16,967	0.0588

The Fit Statistics table, shown above, shows an average squared error of 0.0588 on the VALIDATE partition.

This performance is pretty good, even better than the decision tree tuned in the previous demonstrations. Regardless, try to improve the gradient boosting performance by changing some of the default settings.

5. Close the Results window.

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

6. Reduce **Number of trees** from 100 to **50**.

   Generally, adding more trees to the model can be very slow to overfit. The advice is to keep adding trees until no further improvement is observed.

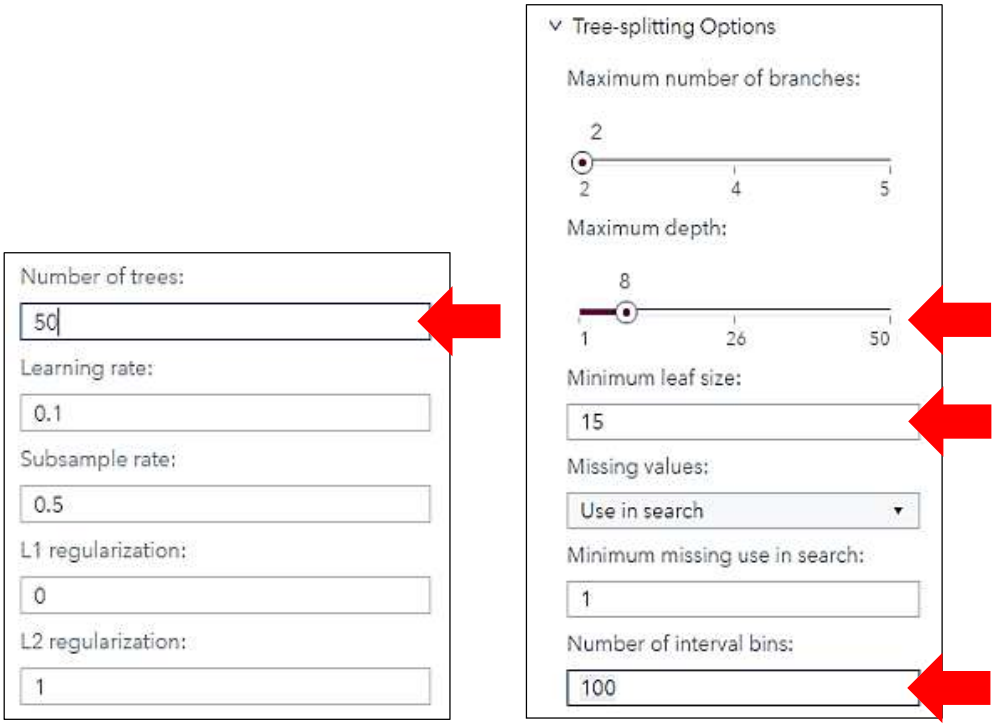
7. Under the Tree-splitting Options properties, increase **Maximum depth** from 4 to **8**.

   Deeper trees are more complex trees and therefore you might prefer shorter trees. Generally, better results are seen with 4 to 8 levels. However, here you try more complex trees to compensate reduced number of trees in the previous setting. It is all trial and error!

8. Increase **Minimum leaf size** from 5 to **15**.

   If you choose too small a leaf size, you can see that it might result in more splits. Too deep a tree means overfitting! On the flip side, if you choose a large leaf size (say in the above example), the tree might stop growing after few splits. ***Note that this might result in poor predictive performance.*** Let's see.

9. Increase **Number of interval bins** from 50 to **100**.



10. Run the **Gradient Boosting** node.

11. Open the results for the node to examine the model fit statistics.

Target ...	Data Role	Partitio...	Formatt...	Sum of ...	Averag...
churn	TRAIN	1	1	39,590	0.0483
churn	VALIDATE	0	0	16,967	0.0569

12. The average squared error for the tuned gradient boosting model is 0.0569 on the VALIDATE partition. This fit statistic is slightly better than the first model, which used the default settings.

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

13. Close the Results window.

End of Demonstration

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.